

**РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ  
ФАКУЛЬТЕТ ФИЗИКО-МАТЕМАТИЧЕСКИХ И  
ЕСТЕСТВЕННЫХ НАУК**

**Лабораторная работа №8.**  
**Программирование цикла. Обработка аргументов**  
**командной строки.**  
Левищева Анастасия Петровна, НКАбд-02-25

# Содержание

|                                        |    |
|----------------------------------------|----|
| Список иллюстраций.....                | 3  |
| 1. Цель работы.....                    | 4  |
| 2. Задание.....                        | 5  |
| 3. Теоретическое введение.....         | 6  |
| 4. Выполнение лабораторной работы..... | 9  |
| 5. Выводы.....                         | 16 |
| Список литературы.....                 | 17 |

## Список иллюстраций

|                                    |    |
|------------------------------------|----|
| Рис.4.1.1.Каталог.....             | 9  |
| Рис.4.1.2.Исполняемый файл(1)..... | 9  |
| Рис.4.3.Изменение(1).....          | 10 |
| Рис.4.1.4.Исполняемый файл(2)..... | 10 |
| Рис.4.1.5.Изменение(2).....        | 10 |
| Рис.4.1.6.Исполняемый файл(3)..... | 10 |
| Рис.4.2.1.Файл lab8-2.asm.....     | 11 |
| Рис.4.2.2.Исполняемый файл(4)..... | 12 |
| Рис.4.2.3.Файл lab8-3.asm.....     | 12 |
| Рис.4.2.4.Исполняемый файл(5)..... | 13 |
| Рис.4.2.5.Изменение(3).....        | 13 |
| Рис.4.2.6.Исполняемый файл(6)..... | 13 |
| Рис.1.Файл var4.asm.....           | 14 |
| Рис.2.Текст программы.....         | 14 |
| Рис.3.Исполняемый файл(7).....     | 15 |

# **1. Цель работы**

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

## 2. Задание

1. Напишите программу, которая находит сумму значений функции  $f(x)$  для  $x = x_1, x_2, \dots, x_n$ , т.е. программа должна выводить значение  $f(x_1) + f(x_2) + \dots + f(x_n)$ . Значения  $x_i$  передаются как аргументы. Вид функции  $f(x)$  выбрать из таблицы 8.1 вариантов заданий в соответствии с вариантом, полученным при выполнении лабораторной работы № 7. Создайте исполняемый файл и проверьте его работу на нескольких наборах  $x = x_1, x_2, \dots, x_n$ .

**Таблица 8.1.** Выражения для  $f(x)$  для задания №1.

| Номер варианта | $f(x)$     | Номер варианта | $f(x)$      |
|----------------|------------|----------------|-------------|
| <b>1</b>       | $2x + 15$  | <b>11</b>      | $15x + 2$   |
| <b>2</b>       | $3x - 1$   | <b>12</b>      | $15x - 9$   |
| <b>3</b>       | $10x - 5$  | <b>13</b>      | $12x - 7$   |
| <b>4</b>       | $2(x - 1)$ | <b>14</b>      | $7(x + 1)$  |
| <b>5</b>       | $4x + 3$   | <b>15</b>      | $6x + 13$   |
| <b>6</b>       | $4x - 3$   | <b>16</b>      | $30x - 11$  |
| <b>7</b>       | $3(x + 2)$ | <b>17</b>      | $10(x - 1)$ |
| <b>8</b>       | $7 + 2x$   | <b>18</b>      | $17 + 5x$   |
| <b>9</b>       | $10x - 4$  | <b>19</b>      | $8x - 3$    |
| <b>10</b>      | $5(2 + x)$ | <b>20</b>      | $3(10 + x)$ |

## 3. Теоретическое введение

### 3.1. Организация стека

Стек — это структура данных, организованная по принципу LIFO («Last In — First Out» или «последним пришёл — первым ушёл»). Стек является частью архитектуры процессора и реализован на аппаратном уровне. Для работы со стеком в процессоре есть специальные регистры (ss, bp, sp) и команды.

Основной функцией стека является функция сохранения адресов возврата и передачи аргументов при вызове процедур. Кроме того, в нём выделяется память для локальных переменных и могут временно храниться значения регистров.

На рис. 8.1 показана схема организации стека в процессоре.

Стек имеет вершину, адрес последнего добавленного элемента, который хранится в регистре esp (указатель стека).

Противоположный конец стека называется дном. Значение, помещённое в стек последним, извлекается первым. При помещении значения в стек указатель стека уменьшается, а при извлечении — увеличивается.

Для стека существует две основные операции:

- добавление элемента в вершину стека (push);
- извлечение элемента из вершины стека (pop).

#### 3.1.1. Добавление элемента в стек

Команда push размещает значение в стеке, т.е. помещает значение в ячейку памяти, на которую указывает регистр esp, после этого значение регистра esp увеличивается на 4. Данная команда имеет один операнд — значение, которое необходимо поместить в стек. Примеры:

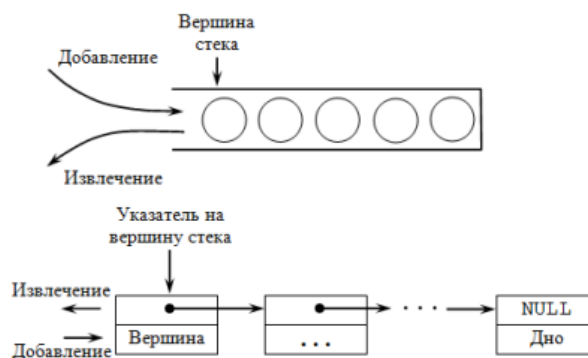


Рис. 8.1. Организация стека в процессоре

```

push -10      ; Поместить -10 в стек
push ebx      ; Поместить значение регистра ebx в стек
push [buf]    ; Поместить значение переменной buf в стек
push word [ax] ; Поместить в стек слово по адресу в ax

```

Существует ещё две команды для добавления значений в стек. Это команда `pusha`, которая помещает в стек содержимое всех регистров общего назначения в следующем порядке: `ax`, `cx`, `dx`, `bx`, `sp`, `bp`, `si`, `di`. А также команда `pushf`, которая служит для перемещения в стек содержимого регистра флагов. Обе эти команды не имеют операндов.

### 3.1.2. Извлечение элемента из стека

Команда `pop` извлекает значение из стека, т.е. извлекает значение из ячейки памяти, на которую указывает регистр `esp`, после этого уменьшает значение регистра `esp` на 4. У этой команды также один операнд, который может быть регистром или переменной в памяти. Нужно помнить, что извлечённый из стека элемент не стирается из памяти и остаётся как “мусор”, который будет перезаписан при записи нового значения в стек.

Примеры:

```

pop eax      ; Поместить значение из стека в регистр eax
pop [buf]    ; Поместить значение из стека в buf
pop word[si] ; Поместить значение из стека в слово по адресу в si

```

Аналогично команде записи в стек существует команда `popa`, которая восстанавливает из стека все регистры общего назначения, и команда `popf` для перемещения значений из вершины стека в регистр флагов.

## 3.2. Инструкции организации циклов

Для организации циклов существуют специальные инструкции. Для всех инструкций максимальное количество проходов задаётся в регистре `ecx`. Наиболее простой является инструкция `loop`. Она позволяет организовать безусловный цикл, типичная структура которого имеет следующий вид:

```
mov     ecx, 100    ; Количество проходов
NextStep:
...
...             ; тело цикла
...
loop    NextStep    ; Повторить `ecx` раз от метки NextStep
```

Инструкция `loop` выполняется в два этапа. Сначала из регистра `ecx` вычитается единица и его значение сравнивается с нулём. Если регистр не равен нулю, то выполняется переход к указанной метке. Иначе переход не выполняется и управление передаётся команде, которая следует сразу после команды `loop`.



## 4. Выполнение лабораторной работы

### 4.1. Реализация циклов в NASM

Создадим каталог для программ лабораторной работы № 8, перейдем в него и создадим файл lab8-1.asm:

```
aplevishcheva@debian:~$ mkdir ~/work/arch-pc/lab08
aplevishcheva@debian:~$ cd ~/work/arch-pc/lab08
aplevishcheva@debian:~/work/arch-pc/lab08$ touch lab8-1.asm
aplevishcheva@debian:~/work/arch-pc/lab08$ ls
lab8-1.asm
aplevishcheva@debian:~/work/arch-pc/lab08$
```

Рис.4.1.1.Каталог

При реализации циклов в NASM с использованием инструкции `loop` необходимо помнить о том, что эта инструкция использует регистр `ecx` в качестве счетчика и на каждом шаге уменьшает его значение на единицу. Внимательно изучим тест листинга 8.1.

Введем в файл `lab8-1.asm` текст программы из листинга 8.1. Создадим исполняемый файл и проверим его работу:

```
aplevishcheva@debian:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
aplevishcheva@debian:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
aplevishcheva@debian:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 10
10
9
8
7
6
5
4
3
2
1
aplevishcheva@debian:~/work/arch-pc/lab08$
```

Рис.4.1.2.Исполняемый файл(1)

Работа файла корректна.

Данный пример показывает, что использование регистра `ecx` в теле цикла `loop` может привести к некорректной работе программы.

Изменим текст программы добавив изменение значения регистра `ecx` в цикле:

```
label:
    sub    ecx, 1
    mov    [N], ecx
```

Рис.4.3.Изменение(1)

Создадим исполняемый файл и проверим его работу:

```
aplevishcheva@debian:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
aplevishcheva@debian:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
aplevishcheva@debian:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 10
9
7
5
3
1
```

Рис.4.1.4.Исполняемый файл(2)

Регистр ecx принимает значения 10, 9, 7, 5, 3, 1. Число проходов цикла не соответствует числу N.

Для использования регистра ecx в цикле и сохранения корректности работы программы можно использовать стек.

Внесем изменения в текст программы добавив команды push и pop (добавления в стек и извлечения из стека) для сохранения значения счетчика цикла loop:

```
label:
    push   ecx
    sub    ecx, 1
    mov    [N], ecx
    mov    eax, [N]
    call   iprintLF
    pop    ecx
    loop   label
```

Рис.4.1.5.Изменение(2)

Создадим исполняемый файл и проверим его работу:

```
aplevishcheva@debian:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
aplevishcheva@debian:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-1 lab8-1.o
aplevishcheva@debian:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 10
9
8
7
6
5
4
3
2
1
0
```

Рис.4.1.6.Исполняемый файл(3)

Число проходов цикла соответствует значению N.

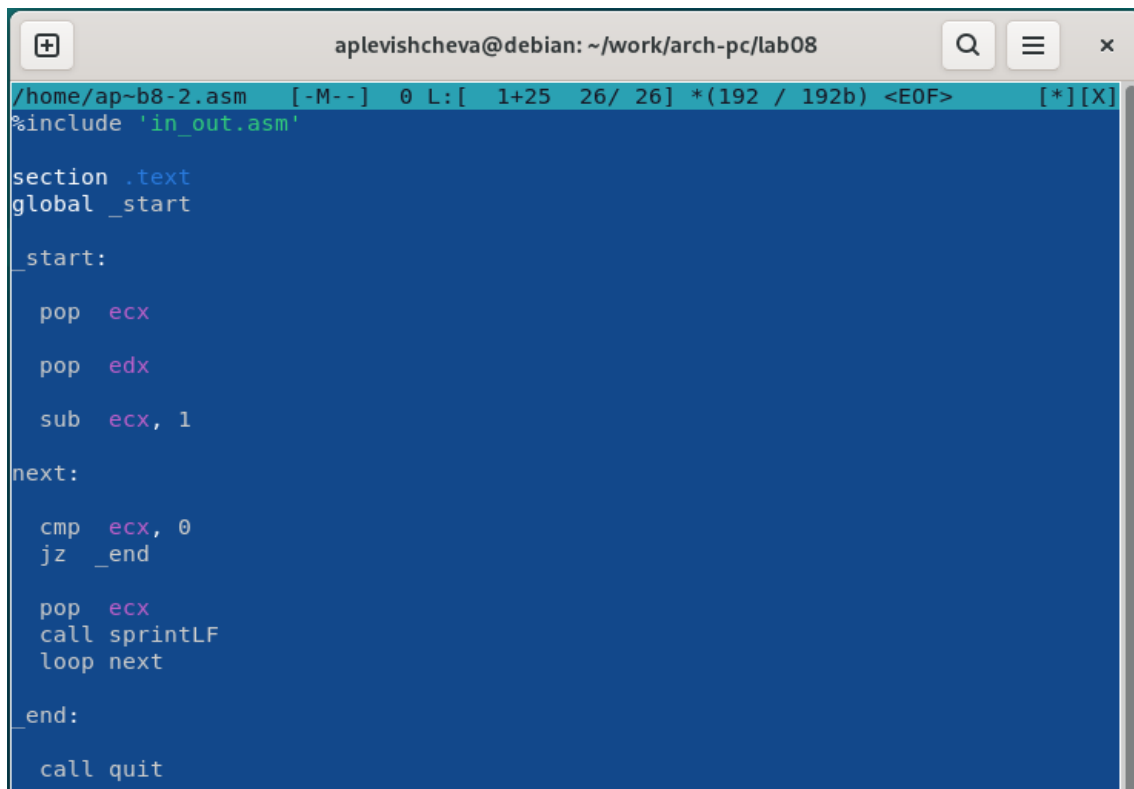
## 4.2. Обработка аргументов командной строки

При разработке программ иногда встает необходимость указывать аргументы, которые будут использоваться в программе, непосредственно из командной строки при запуске программы.

При запуске программы в NASM аргументы командной строки загружаются в стек в обратном порядке, кроме того, в стек записывается имя программы и общее количество аргументов. Последние два элемента стека для программы, скомпилированной NASM, – это всегда имя программы и количество переданных аргументов.

Таким образом, для того чтобы использовать аргументы в программе, их просто нужно извлечь из стека. Обработку аргументов нужно проводить в цикле. Т.е. сначала нужно извлечь из стека количество аргументов, а затем циклично для каждого аргумента выполнить логику программы. В качестве примера рассмотрим программу, которая выводит на экран аргументы командной строки. Внимательно изучим текст программы (Листинг 8.2).

Создадим файл lab8-2.asm в каталоге ~/work/arch-pc/lab08 и введем в него текст программы из листинга 8.2:



```
/home/ap-b8-2.asm [-M--] 0 L:[ 1+25 26/ 26] *(192 / 192b) <EOF> [*][X]
%include 'in_out.asm'

section .text
global _start

_start:

    pop    ecx
    pop    edx
    sub    ecx, 1

next:

    cmp    ecx, 0
    jz     _end

    pop    ecx
    call   sprintf
    loop   next

_end:

    call   quit
```

Рис.4.2.1.Файл lab8-2.asm

Создадим исполняемый файл и запустим его, указав аргументы:

```
aplevishcheva@debian:~/work/arch-pc/lab08$ nasm -f elf lab8-2.asm
aplevishcheva@debian:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-2 lab8-2.o
aplevishcheva@debian:~/work/arch-pc/lab08$ ./lab8-2 аргумент1 аргумент 2 'аргумент 3'
аргумент1
аргумент
2
аргумент 3
aplevishcheva@debian:~/work/arch-pc/lab08$
```

Рис.4.2.2.Исполняемый файл(4)

Программой было обработано 4 аргумента.

Рассмотрим еще один пример программы, которая выводит сумму чисел, которые передаются в программу как аргументы.

Создадим файл lab8-3.asm в каталоге ~/work/arch-pc/lab08 и введем в него текст программы из листинга 8.3.:

```
/home/aplevishcheva@debian:~/work/arch-pc/lab08$ cat lab8-3.asm
[----] 11 L: [ 3+35 38/ 38] *(333 / 333b) <EOF> [*][X]
section .data
    msg db 'Результат: ', 0
section .text
global _start
_start:
    pop ecx
    pop edx
    sub ecx, 1
    mov esi, 0
next:
    cmp ecx, 0h
    jz _end
    pop eax
    call atoi
    add esi, eax
    loop next
_end:
    mov eax, msg
    call sprintf
    mov eax, esi
    call iprintLF
    call quit
```

Рис.4.2.3.Файл lab8-3.asm

Создадим исполняемый файл и запустим его, указав аргументы:

```

aplevishcheva@debian:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
aplevishcheva@debian:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
aplevishcheva@debian:~/work/arch-pc/lab08$ ./lab8-3 12 13 7 10 5
Результат: 47
aplevishcheva@debian:~/work/arch-pc/lab08$ █

```

Рис.4.2.4.Исполняемый файл(5)

Изменим текст программы из листинга 8.3. для вычисления произведения аргументов командной строки:

```

start:
    pop    ecx
    pop    edx
    sub    ecx, 1
    mov    esi, 1
next:
    cmp    ecx, 0h
    jz     _end
    pop    eax
    call   atoi
    imul   esi, eax
    loop   next

```

Рис.4.2.5.Изменение(3)

Создадим исполняемый файл и запустим его, указав аргументы:

```

aplevishcheva@debian:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
aplevishcheva@debian:~/work/arch-pc/lab08$ ld -m elf_i386 -o lab8-3 lab8-3.o
aplevishcheva@debian:~/work/arch-pc/lab08$ ./lab8-3 2 3 4
Результат: 24
aplevishcheva@debian:~/work/arch-pc/lab08$ █

```

Рис.4.2.6.Исполняемый файл(6)

Программа работает корректно.

## Задания для самостоятельной работы.

Задание 1(стр.5)

В ходе лабораторной работы №6 был вычислен мой вариант – 4.

Создадим файл var4.asm в каталоге lab08:

```
aplevishcheva@debian:~/work/arch-pc/lab08$ touch var4-1.asm
aplevishcheva@debian:~/work/arch-pc/lab08$ ls
in_out.asm  lab8-1.asm  lab8-2      lab8-2.o  lab8-3.asm  var4-1.asm
lab8-1      lab8-1.o    lab8-2.asm  lab8-3    lab8-3.o
aplevishcheva@debian:~/work/arch-pc/lab08$
```

Рис.1.Файл var4.asm

Напишем текст программы для вычисления суммы функций:

```
/home/aplevish~ab08/var4.asm  [-M--]  0 L:[ 1+44 45/ 45] *(442 / 442b) <EOF>  [*][X]
#include 'in_out.asm'

section .data
    msg1 db 'Функция: f(x)=2(x-1)', 0
    msg2 db 'Результат: ', 0

section .text
global _start

_start:

    mov     eax, msg1
    call    sprintLF

    pop     ecx
    pop     edx

    sub     ecx, 1
    mov     esi, 0

next:

    cmp     ecx, 0h
    jz      _end

    pop     eax
    call    atoi
    sub     eax, 1
    imul    eax, 2
    add     esi, eax

    loop    next

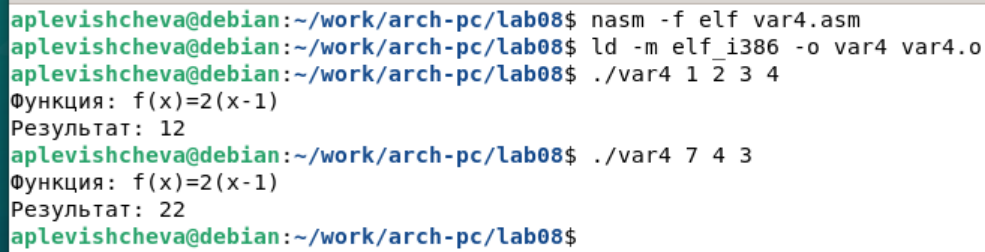
_end:

    mov     eax, msg2
    call    sprint
    mov     eax, esi
    call    iprintLF

    call    quit
```

Рис.2.Текст программы

Создадим исполняемый файл и проверим его работу:



```
aplevishcheva@debian:~/work/arch-pc/lab08$ nasm -f elf var4.asm
aplevishcheva@debian:~/work/arch-pc/lab08$ ld -m elf_i386 -o var4 var4.o
aplevishcheva@debian:~/work/arch-pc/lab08$ ./var4 1 2 3 4
Функция:  $f(x)=2(x-1)$ 
Результат: 12
aplevishcheva@debian:~/work/arch-pc/lab08$ ./var4 7 4 3
Функция:  $f(x)=2(x-1)$ 
Результат: 22
aplevishcheva@debian:~/work/arch-pc/lab08$
```

Рис.3.Исполняемый файл(7)

Проверим правильность выполнения программы аналитически. Результаты совпадают.

## **5. Выводы**

В ходе выполнения лабораторной работы было успешно освоено программирование цикла. Изучена и применена на практике обработка аргументов командной строки.



## Список литературы

1. GDB: The GNU Project Debugger. — URL: <https://www.gnu.org/software/gdb/>.
2. GNU Bash Manual. — 2016. — URL: <https://www.gnu.org/software/bash/manual/>.
3. Midnight Commander Development Center. — 2021. — URL: <https://midnight-commander.org/>.
4. NASM Assembly Language Tutorials. — 2021. — URL: <https://asmtutor.com/>.
5. Newham C. Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 с. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learningbash-Shell-Programming-Nutshell/dp/0596009658>.
6. Robbins A. Bash Pocket Reference. — O'Reilly Media, 2016. — 156 с. — ISBN 978-1491941591.
7. The NASM documentation. — 2021. — URL: <https://www.nasm.us/docs.php>.
8. Zarrelli G. Mastering Bash. — Packt Publishing, 2017. — 502 с. — ISBN 9781784396879.
9. Колдаев В. Д., Лупин С. А. Архитектура ЭВМ. — М. : Форум, 2018.
10. Куляс О. Л., Никитин К. А. Курс программирования на ASSEMBLER. — М. : Солон-Пресс, 2017.
11. Новожилов О. П. Архитектура ЭВМ и систем. — М. : Юрайт, 2016.
12. Расширенный ассемблер: NASM. — 2021. — URL: <https://www.opennet.ru/docs/RUS/nasm/>.
13. Робачевский А., Немнюгин С., Стесик О. Операционная система UNIX. — 2-е изд. — БХВПетербург, 2010. — 656 с. — ISBN 978-5-94157-538-1.
14. Столяров А. Программирование на языке ассемблера NASM для ОС Unix. — 2-е изд. — М. : МАКС Пресс, 2011. — URL: [http://www.stolyarov.info/books/asm\\_unix](http://www.stolyarov.info/books/asm_unix).
15. Таненбаум Э. Архитектура компьютера. — 6-е изд. — СПб. : Питер, 2013. — 874 с. — (Классика Computer Science).
16. Таненбаум Э., Бос Х. Современные операционные системы. — 4-е изд. — СПб. : Питер, 2015. — 1120 с. — (Классика Computer Science).